

STRATA: One Geometry-Free Persistence-and-Periodicity Engine Driving Selectable 2D/3D Lifelong LiDAR Mapping in ROS 2

Jungmo Kang
Independent Researcher
kangjmo91@gmail.com
<https://github.com/kjungmo>

Abstract—A robot that maps the same building for weeks cannot bake every LiDAR hit permanently into one grid: parked carts, passing people, and doors that open on a schedule all burn in, and the map slowly ossifies into a record of everything that was ever seen rather than of what is actually there now. Lifelong mapping must instead hold three properties in tension at once — durability (keep the walls), plasticity (forget the movers), and periodicity (treat a cyclic door as signal, not noise). Per-cell persistence and periodicity models already exist; the closest precedent is Frequency Map Enhancement (FreMen), in the 2D grid of Krajník et al. (2016) and in FROctomap, while several recent LiDAR lifelong-mapping systems (e.g. ELite, LT-mapper) embed persistence logic inside full lifelong-SLAM pipelines. We present STRATA, a small, SLAM-free, geometry-free persistence-and-periodicity engine that combines known ingredients in one shared per-cell state machine — windowed log-odds with survival decay, a Schmitt-trigger graduation band, pruning, and a FreMen-style Fourier test at one configured base period and its harmonics, without period selection — and drives two pluggable geometry backends — a fixed-array 2D occupancy grid and a sparse 3D voxel hash — behind a single interface selected by one runtime string parameter. The only per-backend code is point-to-id mapping and the free-space ray walk; the classifier that graduates, demotes, prunes, and periodicity-labels every cell is shared by composition, not duplicated per dimension. The engine is a plain C++17 + Eigen library with no ROS, DDS, or PCL dependency, exercised by 56 behavior-level tests, so its scientific logic builds and runs deterministically in a second-scale CI loop. A seeded synthetic characterization suite, which uses hand-authored synthetic hit and miss patterns (fed directly to the engine, or as synthetic hit points to the two ray-casting backends; no real or simulated LiDAR scans are processed, and no external baseline, FreMen included, is run), reports that both backends graduate a static wall by the third window and hold recall 1.0 thereafter (their only map-quality divergence, a clutter-induced static-precision gap of 0.9253 vs. 0.9758 under 100 movers per window, arises in the geometry backends’ native sampling, not in the shared classifier); that 50%-duty periodic doors are detected at true-positive rate $2/3$ and false-positive rate $1/5$; that removing hysteresis inflates flicker from 54 to 574 toggles on identical replayed noise (at fixed decay $\lambda = 0.90$); and that, with periodicity off, the estimated memory is a flat ~ 56 B per live cell (evidence record and hash node only) with no per-dimension penalty. Evaluation is synthetic and the tool performs no SLAM; STRATA is the persistence core meant

to sit beneath an external localizer, released open-source as a ROS 2 package.

Index Terms—lifelong mapping, occupancy grids, voxel mapping, persistence filtering, FreMen periodicity, ROS 2, open-source robotics software

I. Introduction

An occupancy grid built the textbook way accumulates evidence and never lets go of it. For a robot that drives a route once, that is fine. For one that maps the same warehouse or hospital corridor for weeks, it is a slow failure: every hit is folded into the same static belief, so a cart parked for an afternoon, a person walking past, and a load-bearing wall all leave the same kind of mark. Over enough passes the map stops describing the building and starts describing the union of everything the sensor ever touched. Suppressing that failure is the whole problem of lifelong mapping, and it forces three requirements that pull in different directions: the map must be durable enough to keep structure that is genuinely permanent, plastic enough to forget structure that has moved on, and must do both without a single global forgetting rate that is simultaneously too slow to erase movers and too fast to trust a wall. This is the stability–plasticity dilemma that Biber and Duckett framed for dynamic maps [1]: represent the environment at more than one timescale at once, or lose either the walls or the ability to adapt.

A third requirement complicates the split further. Some structure is neither permanent nor transient but cyclic — a door open every morning and shut every night, a shutter that tracks business hours. Krajník et al. show that such a cell’s occupancy is a periodic temporal signal, and that a flat decay law erases exactly the regularity that makes it predictable, discarding information a robot could have exploited [2], [3]. A lifelong map therefore needs three coexisting timescales, not two: a fast-fading occupancy signal, a slowly-graduated durable class, and a separately-timed periodic class that a decay term alone would flatten into noise.

Per-cell models of these requirements are not new. Temporal occupancy grids [4] classify cells by their occupancy over several timescales and use the result to locate doors, and Mitsou and Tzafestas [5] keep each cell’s occupancy history to separate static, low-dynamic and high-dynamic cells. The closest prior work is the FreMEn line. Krajník et al. [6] maintain a 2D spatio-temporal occupancy grid that represents the persistence and periodicity of individual cells; it integrates maps that gmapping builds with AMCL pose estimates as odometry, replaces the map server of the ROS navigation stack, and was evaluated on data from several days of routine autonomous patrols of an open-plan office. FROctomap [7] attaches the same spectral model to the voxels of a 3D octree. Several recent LiDAR lifelong-mapping systems instead place persistence logic inside larger pipelines. ELite [8] and LT-mapper [9] resolve durability versus plasticity as parts of full lifelong-SLAM pipelines, coupled to multi-session point-cloud registration, place recognition, and alignment. RTAB-Map [10] offers selectable 2D and 3D geometry under one framework, as a complete SLAM system with loop closure and memory tiering. Khronos [11] generalizes the two-timescale idea to a metric-semantic scene graph, and MTD-Map [12], concurrent with this work, maintains static, dynamic and transition maps from per-voxel statistics of occupancy transitions. Every ingredient STRATA uses therefore has precedent. What we did not find in this literature is their particular combination: one small, SLAM-free implementation that assigns each cell a discrete label through hysteresis graduation and pruning and is shared unchanged by a 2D grid and a 3D voxel hash.

We present STRATA. The name is literal: a cell in the map carries temporal strata — Static, Periodic, and Transient layers over an Unknown baseline — coexisting inside one per-cell state machine, read and written by log-odds occupancy, survival decay, Schmitt-trigger hysteresis, and an incremental Fourier periodicity test. That state machine is geometry-free. Both shipped backends translate world points into int64 cell keys and walk a free-space ray to clear it; everything downstream of the key — accumulation, graduation, demotion, pruning, periodicity labeling — happens once, in the shared engine. The thesis is deliberately narrow: the only per-backend code is point-to-id mapping and the free-space ray walk; the classifier is shared by composition, not duplicated per dimension (Fig. 1). Selecting 2D versus 3D is a single runtime string parameter resolved at construction, not a plugin-loading mechanism, and the engine itself (`strata_core`) is a plain C++17 + Eigen library with no `roscpp`, `tf2`, `PCL`, or `DDS` dependency, so its scientific logic builds and unit-tests without a ROS install.

We are explicit about scope up front (§VI consolidates the non-claims). STRATA is not SLAM: it estimates no pose, closes no loops, and aligns no sessions, consuming instead an external map $\rightarrow$ sensor transform.

Its evaluation is synthetic only — every number below comes from deterministic seeded harnesses and unit tests, not a field deployment, in contrast to the multi-day validations of ELite, LT-mapper, and Berrio et al. [13] — and it runs no external baseline. It models single-session temporal persistence, not multi-session remapping: a graduated cell simply is the current static map, with no session versioning. And its periodicity test checks one configured base period and its harmonics rather than selecting periods from a spectrum as FreMEn does. These are the boundaries of the tool, stated as scope rather than buried as gaps.

Contributions. This paper makes five, each backed only by the shipped code and the synthetic suite:

- 1) One geometry-free engine, two geometries, one switch (by construction). A single int64-keyed persistence-and-periodicity engine (LayeredMap + PeriodicityModel) drives both a fixed-array 2D occupancy grid and a sparse 3D voxel hash behind one MapBackend interface, chosen by a single runtime string parameter, with no state-machine logic duplicated across dimensions (Fig. 1; §III–IV). Per-cell periodic grids already exist in 2D and in 3D [6], [7], as separate implementations [14]; what is added here is one implementation that serves both, and the geometry-free part is the whole persistence, graduation, pruning and classification state machine together with the periodicity test, not per-identifier periodicity modelling alone, which the FreMEn server already provides.
- 2) A ROS-free, deterministically testable core, shown backend-equivalent (design goal + measured). The engine builds and unit-tests with a plain C++17 + Eigen toolchain and is exercised by 56 behavior-level tests; both backends graduate the static wall by window 3 and hold recall 1.0 through window 39, differing in static-layer quality only by a clutter-induced static-precision gap (0.9253 grid2d vs. 0.9758 voxel3d at 100 movers per window) that traces to the backends’ native-geometry sampling, not to the classifier; a separate `per-integrate()` cost gap between the backends is characterized in contribution 5 (§III, §V).
- 3) A minimal lifelong classifier assembled from known parts. Persistence-Filter survival decay [15], Removert-motivated Schmitt hysteresis [16], ReFusion-style negative-evidence ray clearing [17], and a parallel FreMEn-lite periodicity test at a fixed period [2], emitting a four-class label from one dependency-light, SLAM-free module, with every equation transcribed from the shipped code and the implementation-versus-specification differences stated explicitly (§IV–H).
- 4) A mechanistic characterization of the shared classifier (measured, synthetic). 50%-duty periodicity

is detected cleanly (dominant-harmonic amplitude 0.653/0.707, well above the 0.3 threshold) at true-positive rate 2/3 and false-positive rate 1/5, with the single miss diagnosed as a prune/maturity race; and hysteresis is the dominant stabilizer — removing it yields 574 flicker toggles at F1 0.810 versus 54 toggles at F1 1.000 on identical replayed noise at the same decay ($\lambda = 0.90$) (§V).

- 5) A unified engine that adds no per-dimension cost, released as a reproducible tool (measured). Estimated memory is a flat ~ 56 B per cell with periodicity off, and per-window cost tracks live-cell count linearly; the 5.5–15.0 $\times$ per-integrate() gap between backends is explained by 3D free-space voxel proliferation (4.4–10.0 $\times$ more live cells), geometry rather than the engine. The whole system ships as an open-source ROS 2 package with a documented I/O contract and a seeded harness whose figures regenerate from measured CSVs (§III, §V).

On this synthetic suite, the results are consistent with a lifelong-mapping core that is written once and shared across dimensions, with the per-dimension cost arising in each backend’s geometry rather than in the shared engine. Whether STRATA’s periodicity test improves on FreMen’s own component selection, and how the classifier behaves on real LiDAR data, remain to be evaluated (§VI).

STRATA is available at github.com/kjungmo/strata as a ROS 2 Humble package under an open-source license, with a thin ROS adapter node and a reusable characterization harness; it is designed to plug beneath an external localizer such as `prism_loc`. The remainder of the paper positions STRATA against the prior art (§II), specifies its two-band, testable architecture (§III), gives the as-shipped engine mathematics including three honest implementation-versus-specification differences (§IV), reports the synthetic evaluation (§V), and states the limitations, boundaries, and reproducibility of the tool (§VI–§VII).

II. Related Work

We organize prior work into six themes and, for each entry, state STRATA’s exact relation rather than a generic contrast.

Lifelong and persistent mapping. ELite [8] and LT-mapper [9] are recent LiDAR lifelong-mapping systems with the same aim as STRATA. ELite computes a two-timescale ephemerality score per point (within-session dynamic vs. across-session transient) and drives a Life-long/Static/Delta map triad; LT-mapper composes a live map, a meta map, and a delta map through explicit LT-SLAM alignment, LT-removert removal, and LT-map graduation stages. Both are full lifelong-SLAM pipelines with multi-session point-cloud registration and place recognition. STRATA implements only the graduation half of that pattern — one per-cell state machine (Static/Periodic/Transient/Unknown, §IV-E) inside

a ROS-free engine, with no scan matching, no loop closure, no cross-session alignment, and no semantics. STRATA is not a competing full system; it is the persistence-and-classification core that ELite’s triad and LT-mapper’s live-to-static pipeline wrap around, and its measured evaluation (§V) is correspondingly narrower — a synthetic, single-session characterization, not the real-data and field-scale validation ELite and LT-mapper report. Khronos [11] generalizes the same two-timescale idea to an active-window/long-term-reconstruction split with semantics and a scene graph; STRATA stays at the raw per-cell level with no object or scene-graph layer, a narrower and cheaper instance of the same principle. Classifying cells by their temporal behaviour predates these systems. Temporal occupancy grids [4] classify cells by occupancy over several timescales with planar laser range-finders and locate doors in a real-world setting; Mitsou and Tzafestas [5] index every cell’s occupancy history over time and separate static, low-dynamic and high-dynamic cells, in simulation with known poses. Biber and Duckett [1] and Meyer-Delius et al. [18] contribute the multi-timescale and static/temporary-split ideas STRATA’s window/decay/hysteresis/periodicity combination descends from. RTAB-Map [10] offers selectable 2D/3D geometry inside one shipped framework, coupled to full SLAM and STM/WM/LTM memory management; STRATA narrows this to the mapping/persistence layer alone, behind a MapBackend interface selected by a single string parameter, consuming an externally supplied pose rather than estimating one. Berrio et al. [13] report an 18-month field deployment of the same purge/promote pattern STRATA implements as a Schmitt trigger (§IV-C) — field-scale evidence for the pattern that STRATA currently lacks for itself, and Pomerleau et al. [19] infer online whether each 3D point is static or dynamic from repeated observations, over seven months of data. Yang et al. [20] keep positive/negative version deltas over a base map so any past session can be reconstructed; STRATA does no session versioning at all — a graduated cell is simply part of the current static map, its history implicit in log-odds and observation counts, not explicitly replayable. For 2D occupancy grids, Stefanini et al. [21], [22] update a LiDAR occupancy map for long-term operation while accounting for localisation error, with the aim of keeping the map current rather than modelling periodic change. Two 2026 works, concurrent with this one, produce layered outputs: MTD-Map [12] encodes the direction and duration of occupancy transitions per voxel in one stage and outputs static, dynamic and transition maps, evaluated on real datasets against established baselines, and Chen and Sun [23] assign map content to long-term static, potentially dynamic and removed observed-dynamic layers from geometric and semantic evidence. Neither describes an explicit periodic class.

Persistence and forgetting. The Persistence Filter [15] recursively estimates each feature’s survival probability

from a Bayesian survival-time model. STRATA’s per-window survival-decay multiplier λ (§IV-B) is a discretized, single-scalar simplification of that idea: one tunable multiplicative factor pulls belief toward unknown each window in place of a full survival-time posterior. The log-odds hit/miss update with clamping in Thrun et al. [24] is used verbatim as STRATA’s per-window occupancy accumulation, with decay and hysteresis layered on top. Tipaldi et al. [25] and Saarinen et al. (iMac) [26] model each cell as a two-state Markov process with a recency-weighted or online-learned transition rate — the principled, per-cell-learned form of “how fast a cell should forget.” Perpetua [27] combines persistence and emergence filters under multiple hypotheses for features that disappear and reappear. `survival_decay` is a single global constant, a constant-rate special case of that formalism; STRATA states this as a documented simplification (§VI), not an unacknowledged gap.

Periodicity. FreMen [2] is the direct counter-argument STRATA’s Periodic class answers: a cell’s occupancy can be a periodic temporal signal, and a flat decay erases exactly that structure. FreMen models cyclic occupancy with a few Fourier components selected by amplitude from a set of candidate periods, and it is the closest prior work to STRATA. Krajník et al. [6] use it in a 2D spatio-temporal occupancy grid in which each cell represents both persistence (through the mean time between state changes) and periodicity; the grid integrates one map per patrol, replaces the ROS map server, and predicts time-specific maps for localisation and planning. FROctomap [7] applies the same model to 3D octree voxels, the exploration work of Krajník et al. [28] displays the static and daily-periodic cells of such a grid, and the open-source FreMen repository [14] provides separate 2D-grid and octree packages, and also a generic server that maintains FreMen models of arbitrary binary states keyed by an identifier. Warped hypertime [29] extends the idea to pseudo-periodic variation. STRATA is narrower than this family in period modelling: it tests one configured base period and its harmonics and does not select periods. It differs from the family in its primary output, a discrete class reached through a hysteresis band and pruning rather than a predicted occupancy probability, and it updates once per window, whereas the grid of Krajník et al. [6] integrates one map per patrol. STRATA’s PeriodicityModel (§IV-D) runs FreMen’s incremental-Fourier accumulation in parallel with decay and graduation, so an oscillating cell is classified Periodic rather than mis-graduated to Static or mis-pruned as Transient — the third class exists specifically to answer this argument, at the cost of a bounded harmonic count and a validity gate tied to touched-window count rather than elapsed time (a documented implementation-versus-specification divergence, §IV-H). Krajník et al. [3] is the earlier, more general spectral-analysis lineage FreMen later packaged as a mapping method; STRATA’s bounded incremental-

Fourier approximation is a further narrowing of that lineage, not the full spectral machinery.

Dynamic-object removal. Removert [16] conservatively removes dynamic points and then reverts wrongly removed static ones, because aggressive removal erases real structure. This is the exact justification for STRATA’s hysteresis band (§IV-C): a graduated cell demotes only when its probability falls below $p_{\text{dem}} < p_{\text{grad}}$, i.e. sustained contradicting evidence across windows rather than a single contradicting frame — the same failure mode Removert’s revert pass patches after the fact, STRATA prevents online by construction. ReFusion [17] treats a reliably observed empty voxel as evidence, not merely an absence of evidence; STRATA’s ray-clearing — exact Bresenham for grid2d, sub-voxel-step sampling for voxel3d (§IV-F) — applies the same negative-information logic online, with misses accumulating as l_{miss} and eventually demoting a graduated cell once an object vacates it. ERASOR [30] removes dynamic points from an already-accumulated 3D map offline via per-bin pseudo-occupancy ratios; STRATA never batches and performs no object-level reasoning — every cell’s class is a running online per-window hit/miss-and-decay state, the baseline ERASOR’s batch, object-level approach is not attempting to be.

Production stacks. OctoMap [31] shares STRATA’s log-odds-with-clamping core but compresses storage with an octree; voxel3d instead uses a flat int64-keyed hash for $O(1)$ insert/lookup, trading multi-resolution compression for simplicity and unit-testability (§VI). The voxel-hashing scheme of Nießner et al. [32] is the direct data-structure this hash follows. UFOMap [33] makes “never observed” an explicit third per-voxel state; STRATA gets the same distinction implicitly from the observations counter and log-odds value rather than a dedicated state, cheaper to implement and less explicit to query. Nav2 STVL [34] decays costmap obstacle evidence for short-horizon planning using the same decay-rate mechanism family STRATA’s `survival_decay` uses to decay mapping evidence toward unknown — same knob, different consumer. Layered Costmaps [35] composite independently maintained plugin layers per cell; STRATA’s output value convention (static→100, periodic→75, transient→50, free→0, unknown→−1) reproduces the same layered mental model from one cell-level state machine, plus a per-cell byte for the free and last-observation values, rather than from multiple composited layers. SLAM Toolbox [36] bounds compute by adding and removing pose-graph nodes while performing its own 2D SLAM; STRATA has no pose graph and no SLAM, consuming an externally supplied transform and pruning at the per-cell evidence level (§IV-E) instead — a different layer of the stack, included as the standard 2D lifelong-mapping comparison point.

Pluggable-backend architecture. ROS 2 Pluginlib [37] is the canonical swappable-backend mechanism — abstract base class, XML plugin description, runtime dlopen-based

loading — and the pattern STRATA deliberately rejects, not adopts. With exactly two backends, STRATA resolves MapBackend to one of two unique_ptrs at construction from a single runtime string parameter (§III); no plugin XML, no dynamic loading, no ament_index registry. This is the right choice at two backends; pluginlib becomes the right choice only once backend count grows past what a single if/else factory reads clearly. RTAB-Map [10] is recited here as a shipped system offering selectable 2D/3D geometry under one framework, narrowed by STRATA to a persistence-only module with the classifier shared by composition rather than duplicated per geometry — the only per-backend code is point-to-CellId mapping and the free-space ray walk.

Relative to the works above, the structural difference is narrow. Periodicity modelling that does not depend on geometry already exists, since the FreMEEn server models any binary state that has an identifier [14]. What STRATA shares, unchanged, between its 2D and 3D ray-casting backends is the full cell-level state machine: windowed log-odds with survival decay, graduation and demotion hysteresis, pruning, layer classification and the periodicity test.

III. System Overview and Architecture

STRATA is delivered as two ament packages with a deliberate dependency boundary between them. strata_core is the scientific engine: pure C++17 with Eigen as its only third-party dependency, holding the entire occupancy/persistence/periodicity state machine and both geometry backends. strata is a thin ROS 2 node that wraps the engine, owning message conversion, TF lookups, publishers, subscribers, services, and file I/O. Every ROS-only dependency — rclcpp, tf2, sensor_msgs, nav_msgs, PCL — lives exclusively in strata; strata_core links none of them. This split is the reason the engine builds and unit-tests with a plain system toolchain (cmake -S strata_core -B build -DSTRATA_CORE_BUILD_TESTS=ON && ctest), with no ROS install, no colcon, no DDS discovery, and no TF buffer warm-up in the loop. The architecture is shown in Fig. 1.

A. The MapBackend interface and the geometry-free invariant

strata_core::MapBackend is a pure abstract base with four methods:

```
struct Observation { std::vector<Eigen::Vector3d> hits; }; //
  endpoints, MAP frame
class MapBackend {
  virtual void integrate(const Observation& obs,
                        const Eigen::Vector3d&
                        sensor_origin_map) = 0;

  virtual bool tick() = 0;
  virtual std::size_t staticCellCount() const = 0;
  virtual std::size_t transientCellCount() const = 0;
};
```

Observation::hits is a flat list of 3D endpoints already expressed in the map frame; producing it from a

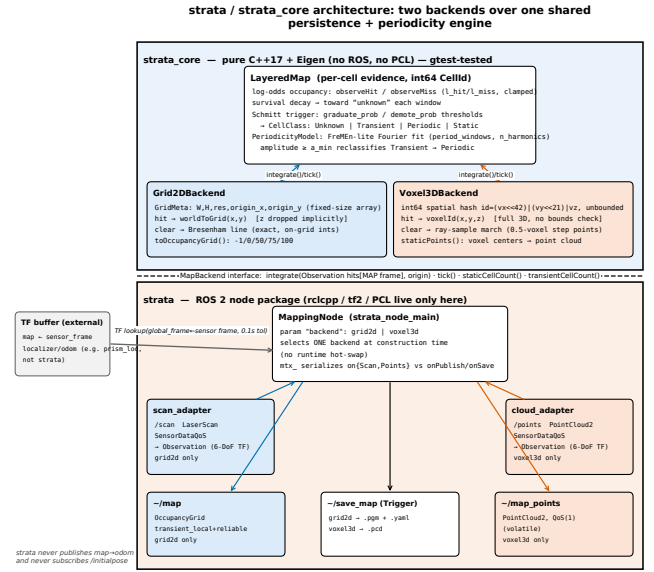


Fig. 1. Two-band architecture of STRATA. The upper band is strata_core (pure C++17 + Eigen, no ROS or PCL, gtest-tested): one shared LayeredMap persistence-and-periodicity engine, keyed by int64 CellId, feeds two sibling backend boxes Grid2DBackend and Voxel3DBackend. The lower band is the strata ROS 2 node, where MappingNode selects one backend at construction, its scan/cloud adapters convert messages to Observations in the map frame, and its timer publishes ~map / ~map_points and services ~save_map. The two bands are split by the MapBackend interface line. The external TF arrow (map → sensor frame) enters the node from outside the diagram: STRATA consumes localization but never produces it.

sensor message and a sensor_to_map transform is the caller’s job, so the backend never touches TF. integrate() performs one discrete map update from one sweep or cloud: for each hit it clears the free-space cells between sensor_origin_map and the hit via a backend-specific ray operation, then registers the hit itself as an occupied observation. tick() advances the temporal window and returns whether a window boundary was crossed. staticCellCount() and transientCellCount() are read-only introspection forwarded to the shared classifier.

Both concrete backends hold a LayeredMap member and satisfy the interface purely by translating world-frame geometry into int64 CellId keys that LayeredMap accumulates evidence for. This is the load-bearing design invariant: the only per-backend code is point→id mapping and the free-space ray walk; the occupancy/persistence/periodicity classifier is shared by composition, not duplicated per dimension. LayeredMap and PeriodicityModel hold no coordinates, no resolution, and no origin — only integer ids — so the identical engine drives both 2D and 3D. The consequence, which the evaluation returns to, is that any cross-backend behavioral divergence or cost gap must originate in the geometry layer, because that is the only layer that differs.

TABLE I
ROS 2 input/output contract of the strata node.

Dir.	Name	Type / QoS	Condition
Sub	scan_topic (/scan)	LaserScan, Sensor-Data	grid2d only
Sub	points_topic (/points)	PointCloud2, SensorData	voxel3d only
Pub	~/map	OccupancyGrid, QoS(1) TL reliable	grid2d only
Pub	~/map_points	PointCloud2, QoS(1) volatile	voxel3d only
Srv	~/save_map	Trigger, default	both (.pgm+.yaml / .pcd) drives publishing, both
Timer	publish_period (1.0 s)	—	both; one per second (wall clock)
Pub	/diagnostics	diagnostic_msgs/DiagnosticArrayinput-rate depth 10	status per second (wall clock)

B. Backend selection

Selection is a single string ROS parameter, backend ("grid2d" or "voxel3d"), read once in the node constructor. It is not polymorphic dispatch over a MapBackend*: the node holds both `std::unique_ptr<Grid2DBackend>` and `std::unique_ptr<Voxel3DBackend>` as separate members, and an if/else at construction instantiates exactly one of them, leaving the other null, and wires up the backend-specific parameters, publisher, and subscriber. We use a compile-time interface plus one runtime switch rather than dlopen-based plugin loading. At two backends, pluginlib [37] would add a registration and discovery mechanism whose cost is not repaid until the backend count grows; we defer that machinery until it is. Both branches read every LayeredMapParams field through a common helper, so the persistence and periodicity tuning surface is backend-independent by construction.

C. ROS 2 I/O contract

The node is `strata::MappingNode`, default node name `strata`. It looks up `global_frame` (default map) $\leftarrow$ sensor frame on every incoming scan or cloud, at the message stamp, as a full 6-DoF `Eigen::Isometry3d`; there is no yaw-only flattening on the input side. Lookup failures are caught and rate-throttled rather than fatal, and that scan or cloud is dropped. STRATA is mapping-only: it neither subscribes to `/initialpose` nor broadcasts `map` $\rightarrow$ `odom`. Pose must already be published on TF by an external localizer. A single mutex serializes integration against publish and save.

The occupancy-value convention on `~/map` and saved files is: unknown -1 , free 0 (a ray has cleared the cell and no higher value claims it), transient 50 , periodic 75 , static 100 (rendering rule in §IV-F). A saved PGM writes $0 \rightarrow 254$, 50 and $75 \rightarrow 100$, $100 \rightarrow 0$ and $-1 \rightarrow 205$, so with

the thresholds written beside it `nav2_map_server` loads free cells as free, static cells as occupied, and transient and periodic cells as unknown. For `voxel3d`, only the static-cell set is emitted as points on `~/map_points`.

IV. The Layered Map Engine

This section specifies the engine as implemented in `strata_core`. All equations are transcribed from the source, not from the design specification; where the two disagree, §IV-H documents the difference. Symbols follow the notation used throughout the paper: ℓ is a cell's log-odds occupancy evidence, $p = \sigma(\ell)$ its occupancy probability, t the window (phase) index, H the harmonic count, T the base period, and λ the survival decay.

A. Per-frame accumulation and windowing

Each `observeHit(id) / observeMiss(id)` only increments the cell's window counters, `window_hits` or `window_misses`; it performs no log-odds arithmetic. Cells are created lazily on first touch, with $\ell = 0$, `observations=0`, and `graduated=false`. `tick()` advances the integration counter and closes a window on the interval boundary:

$$\text{closeWindow} \iff (\text{layer_interval} \leq 1) \vee (\text{integration_count} \bmod \text{layer_interval} = 0). \quad (1)$$

The layered update, `endWindow()`, then runs once over all live cells. With window counts h_w and m_w , each cell's window is scored:

$$\begin{aligned} \text{touched} &= [h_w > 0 \vee m_w > 0], & \text{occ} &= [h_w > 0], \\ \text{free} &= [h_w = 0 \wedge m_w > 0]. \end{aligned} \quad (2)$$

A single hit outweighs any number of misses in the same window.

The engine counts windows in integration ticks. The ROS node can instead close windows on message time (`window_mode` time, which the shipped parameter files select; the code default and every experiment in §V use scans): a window then spans `window_period_s` seconds from the first message's stamp, computed in integer nanoseconds, and a message stamped past the open window first closes it and every window since that saw no message (`closeWindows(k)`). A window that observes nothing changes no cell, so closing k windows is exactly equivalent to k calls of `endWindow()` (a unit test checks it) yet costs one pass: the first close settles every cell and the rest only advance the window index. The clock re-anchors when a stamp falls more than half a window behind the newest one (a bag loop) or lands more than 10^6 windows ahead (an unset stamp). Silent windows advance the phase without a touch, and the touched windows stay independent of occupancy as long as message loss is, so the false-alarm bound of §IV-D holds in either mode; what the window rule changes is the period, in seconds, that `period_windows` stands for, and with it the test's power.

With scan-counted windows a window lasts `layer_interval` divided by the rate of messages actually integrated, so lost messages stretch it (§VI).

B. Log-odds persistence with survival decay

For touched cells only, the engine applies one log-odds increment (an inverse-sensor-model update in the sense of [24]), multiplies by the survival decay λ , and clamps:

$$\ell \leftarrow \text{clamp}\left(\lambda(\ell + \underbrace{[\text{occ}] l_{\text{hit}} + [\text{free}] l_{\text{miss}}}_{\text{one increment}}), l_{\min}, l_{\max}\right), \quad (3)$$

$$\text{clamp}(x, a, b) = \min(b, \max(a, x)), \quad (4)$$

with occupancy probability $p = \sigma(\ell) = 1/(1 + e^{-\ell})$. The survival multiplier is a constant-rate special case of the Persistence Filter forgetting model [15].

Ordering note (load-bearing). The decay multiplies the already-incremented value, so the fresh hit or miss is itself attenuated by λ in the same window. This is not the classic decay-then-add order. One consequence is the decayed fixed point of repeated hits, $\ell^* = \lambda l_{\text{hit}}/(1 - \lambda) \approx 27.5$, which the clamp caps at $l_{\max} = 5$, giving $p_{\max} = \sigma(5) \approx 0.9933$. With the defaults, $p_{\text{grad}} = 0.8$ corresponds to $\ell \geq \ln 4 \approx 1.386$ and $p_{\text{prune}} = 0.05$ to $\ell < \ln(1/19) \approx -2.944$.

C. Schmitt-trigger graduation and demotion

After the log-odds update, p and the periodicity amplitude a (§IV-D) are recomputed for every cell, touched or not, and the graduated flag g is updated by a two-sided Schmitt trigger:

$$\begin{aligned} \text{graduate: } & \neg g \wedge \neg \text{periodic} \wedge p \geq p_{\text{grad}} \wedge \\ & \text{observations} \geq N_{\min} \Rightarrow g \leftarrow \text{true}, \end{aligned} \quad (5)$$

$$\text{demote: } g \wedge (p \leq p_{\text{dem}} \vee \text{periodic}) \Rightarrow g \leftarrow \text{false}. \quad (6)$$

The interval $[p_{\text{dem}}, p_{\text{grad}}]$ is the hysteresis band that suppresses flicker at the static boundary; the motivation is the same observation-consistency argument as in dynamic-object removal by reverting [16]. Two periodicity guards go beyond a bare threshold rule: !periodic blocks a strongly periodic cell from ever graduating to Static, and || periodic force-demotes a graduated cell that later reveals periodicity. The two rules cannot fight in one window, because graduation requires $\neg \text{periodic}$ and $p \geq p_{\text{grad}} > p_{\text{dem}}$.

D. FreMEn-lite periodicity

In parallel with occupancy, each touched cell (when `enable_periodicity`) feeds an incremental Fourier model in the spirit of FreMEn [2]. The model is restricted to one configured base period and its first harmonics; unlike FreMEn, it does not select periods from a candidate set, so a cycle whose frequency is not a harmonic of the base frequency is outside its scope. With the per-window

occupancy sample $v = [\text{occ}] \in \{0, 1\}$ and phase index t , gather accumulates:

$$n += 1, \quad S_0 += v, \quad (7)$$

$$\begin{aligned} C_k &+= v \cos((k+1)\omega t), \\ S_k &+= v \sin((k+1)\omega t), \quad k = 0..H-1, \end{aligned} \quad (8)$$

$$\omega = \frac{2\pi}{\max(1, T)}. \quad (9)$$

Here n counts touched windows fed to gather, not elapsed windows. The phase prediction at window t is

$$\hat{p}(t) = \text{clip}_{[0,1]} \left(\frac{S_0}{n} + \sum_{k=0}^{H-1} \left[a_k \cos((k+1)\omega t) + b_k \sin((k+1)\omega t) \right] \right), \quad (10)$$

with DC term equal to the empirical mean S_0/n and harmonic coefficients $a_k = 2C_k/n$, $b_k = 2S_k/n$. The dominant-harmonic amplitude and the periodicity test are

$$a = \begin{cases} 0, & n < T \\ \max_{0 \leq k < H} \sqrt{a_k^2 + b_k^2}, & n \geq T \end{cases} \quad (11)$$

$$\text{periodic} = \text{enable_periodicity} \wedge a \geq a_{\min}. \quad (12)$$

The $n \geq T$ gate means a sparsely observed cell needs T touches before its amplitude is trusted, which can span far more than T elapsed windows.

E. Cell-class state machine and pruning

At the end of each window, cells below confidence are erased:

$$\text{erase cell} \iff \neg g \wedge p < p_{\text{prune}} \wedge a < a_{\min}. \quad (13)$$

Static cells (g) and periodic cells ($a \geq a_{\min}$) are never pruned; pruning a cell also erases its FreMEn coefficients. A snapshot classifier reads out one of four states by a priority ladder:

$$\begin{aligned} & \text{absent} \rightarrow \text{Unknown}; \quad g \rightarrow \text{Static}; \\ & (\text{enable} \wedge a \geq a_{\min}) \rightarrow \text{Periodic}; \quad p \geq p_{\text{prune}} \rightarrow \text{Transient}; \\ & \text{else} \rightarrow \text{Unknown}. \end{aligned} \quad (14)$$

The full transition table is Table II. Note that a present cell classifies Unknown when $p < p_{\text{prune}}$ and it is neither periodic nor graduated — the same condition under which it is generally pruned in the same window.

The complete per-window update is Algorithm 1.

TABLE II
Cell-class transitions, all evidence-driven and evaluated at window close.

From	To	Trigger
(implicit) Unknown	Transient	first observeHit/Miss: cell created, $\ell=0 \Rightarrow p=0.5 \geq p_{\text{prune}}$
Transient	Static	$p \geq p_{\text{grad}} \wedge \text{obs} \geq N_{\text{min}} \wedge \neg \text{periodic}$ (graduate)
Transient	Periodic	$a \geq a_{\text{min}}$ (needs $n \geq T$ touched windows)
Transient	Unknown (erased)	$p < p_{\text{prune}} \wedge a < a_{\text{min}} \wedge \neg g$ (prune)
Static	Transient / Periodic / Unknown	demote ($p \leq p_{\text{dem}} \vee \text{periodic}$), then re-classified by ladder
Periodic	Transient / Unknown	a falls below a_{min} (then subject to prune)
Periodic	—	never graduates (!periodic guard), never pruned while $a \geq a_{\text{min}}$
Static	—	never pruned while g

Algorithm 1 endWindow(): one layered update per closed window, over all live cells

```

for each live cell c: # main update pass
  touched <- [h_w>0 or m_w>0]
  occ <- [h_w>0]
  free <- [h_w=0 and m_w>0]
  if touched:
    l <- 1 + [occ]*l_hit + [free]*l_miss # one log-odds
    increment
    l <- lambda * l # decay attenuates the fresh increment
    l <- clamp(l, l_min, l_max)
    observations <- observations + 1
    if enable_periodicity:
      gather(c, occ, t) # n+=1; S0+=v; Ck+=v cos((k+1)
        w t); Sk+=v sin((k+1)w t)
  p <- sigma(l); a <- amplitude(c) # recomputed for ALL
    cells, touched or not
  periodic <- enable_periodicity and a >= a_min
  if not g and not periodic and p >= p_grad and observations
    >= N_min:
    g <- true # graduate -> Static
  else if g and (p <= p_dem or periodic):
    g <- false # demote
  reset h_w <- 0, m_w <- 0
  for each live cell c: # prune pass
    if not g and p < p_prune and a < a_min:
      erase c and its FrEMEn coefficients
  t <- t + 1

```

The window update is $O(N \cdot H)$ for N live cells and H constant, hence $O(N)$; it iterates all cells, including untouched ones, because the Schmitt and prune decisions read the recomputed p and a of every cell. Between window closes a frame is $O(1)$ per endpoint plus the backend ray cost.

F. Geometry backends

The two backends differ only in how a world point becomes a CellId and how the free-space ray is walked; both delegate all evidence updates to the shared LayeredMap.

Grid2DBackend addresses a fixed-size, fixed-origin 2D array described by GridMeta = {width, height, resolution, origin_x, origin_y}. It buckets each hit’s (x, y) (dropping z) into a row-major id $\text{gridCellId}(m, g_x, g_y) = g_y \cdot \text{width} + g_x$; points outside the array are dropped. Free space is cleared with integer Bresenham’s line algorithm [38] from the sensor cell up to but excluding the hit cell, calling observeMiss on every cell in that half-open span — including the sensor-origin cell — so the hit cell receives only observeHit. Outside the classifier, the backend keeps

one byte per grid cell: an “observed free” bit set by the ray clear and never cleared, a bit marking a hit in the open window, and a bit recording whether the last closed window that observed the cell saw a hit (a window that both hits and clears a cell counts as a hit, as in the classifier). Rendering to nav_msgs/OccupancyGrid initializes every cell to -1 , sets every observed-free cell to 0, then overwrites in ascending priority transient obstacles $\rightarrow 50$ (a cell hit in the open window or last observed as a hit, or a Transient cell whose evidence leans occupied, $p > 0.5$), periodic $\rightarrow 75$, static $\rightarrow 100$, so static wins any tie. A new object on long-cleared ground therefore reads 50 from its first hit even while its log-odds are still negative, and keeps reading 50 while it is out of view until a ray observes it free again; a cell last observed free that does not lean occupied reads 0, including after the classifier prunes it. Free thus means “last observed free”, not “currently free”.

Voxel3DBackend is a sparse hash: the map grows through the hash map and needs no preallocated extent. Each axis is floor-divided by voxel_size and offset by $k\text{Off} = 1 \ll 20$ to keep the per-axis index non-negative, and the three 21-bit ($k\text{Bits} = 21$) fields are packed into one 64-bit key, $\text{id} = (v_x \ll 42) | (v_y \ll 21) | v_z$; voxelCenter is the exact inverse, returning $(v + 0.5) \cdot \text{voxel_size}$ per axis. This gives $O(1)$ hashing. The key space is nevertheless finite: each 21-bit field represents the indices -2^{20} to $2^{20} - 1$ around the origin, far beyond the extent of a building at the default voxel size, and the code does not check this range, so a point outside it would receive a wrong key rather than being dropped. Free space is cleared by ray-sample marching, not geometric voxel traversal: the ray is subdivided into $\lfloor \text{len} / (0.5 \cdot \text{voxel_size}) \rfloor$ half-voxel steps, and each interior sample is hashed to its containing voxel and marked observeMiss. This is simpler than exact traversal but can over-sample a long ray and can skip a thin voxel when the step count rounds down. The hit is always observeHit, with no bounds check: the hash grows to fit, and the key range is not checked (above). staticPoints() maps the static-cell set through voxelCenter to a point cloud. Table III contrasts the two.

G. Parameters

Table IV lists the engine’s tunable parameters with their defaults, which are identical between the struct definition

TABLE III

The two geometry backends. All persistence and periodicity logic is shared; only these two rows of behavior differ.

	Grid2DBackend	Voxel3DBackend
Cell addressing	2D array index via GridMeta (fixed $W \times H$, fixed origin)	3D spatial hash, sparse, int64 packed key (21 bits per axis)
Hit dimensionality	x, y (z dropped by projection)	x, y, z (full volumetric)
Free-space ray	Bresenham line (exact, on-grid)	ray-sample march at 0.5-voxel step (approximate, off-grid)
Growth	none — pre-sized at construction	sparse — hash grows; per-axis index range fixed by the 21-bit key
Output	nav_msgs/OccupancyGrid (dense $-1/0/50/75/100$)	static voxel centers $\rightarrow$ PointCloud2

TABLE IV

Engine parameters and defaults (struct and params/*.yaml agree).

Name	Default	Meaning
layer_interval	10	integration ticks per layered-update window
l_hit	0.85	log-odds increment on an occupied window
l_miss	-0.4	log-odds increment on a free window
l_min	-5.0	clamp lower bound (log-odds)
l_max	5.0	clamp upper bound (log-odds)
survival_decay (λ)	0.97	Persistence-Filter forgetting multiplier, per window
graduate_prob	0.8	p to promote $\rightarrow$ Static
(p_{grad}) demote_prob	0.45	p at/below which Static demotes (hysteresis floor)
(p_{dem}) min_observations	3	min touched windows before a cell may graduate
(N_{min}) prune_prob	0.05	erase non-static, non-periodic cell below this p
(p_{prune}) enable_periodicity	true	run the FreMEn model
periodic_amplitude_min (a_{min})	0.3	dominant-harmonic magnitude to classify Periodic
period_windows	24	FreMEn base period; also amplitude-validity gate ($n \geq T$)
(T) n_harmonics (H)	2	Fourier harmonics tracked per cell

and both shipped YAML files. Geometry and node parameters (grid_width 400, grid_height 400, grid_resolution 0.05 m, grid_origin_{x,y} -10.0 m for grid2d; voxel_size 0.2 m for voxel3d; frame names, topics, publish_period 1.0 s; the window clock window_mode (scans in code, time in the shipped files) and window_period_s 1.0 s; the rate check expected_scan_rate_hz 0, meaning estimated, and rate_warn_drop_fraction 0.05 and input_timeout_s 5 s) are not part of the engine math.

Per-cell state is compact: CellEvidence is 24 bytes, plus roughly 32 bytes of unordered_map node overhead, giving the ~ 56 B/cell footprint reported in the evaluation. That figure is an estimate for the periodicity-off path, not a measurement. A FreMEn-tracked cell adds a Coeff record (~ 96 B payload at $H = 2$) in a second hash map, allocated only for cells touched at least once with periodicity enabled; it is not included in the estimate, nor are the hash tables' bucket arrays. grid2d also keeps the flag byte per grid cell (observed-free and last-observation bits; width $\times$ height bytes, the size of the published OccupancyGrid) and the ids touched in the open window, outside the per-cell estimate.

H. Implementation versus specification

We document the code that ships, not the design prose. Three points where the implementation diverges from SPEC.md are surfaced here rather than smoothed over, because each changes the operational semantics.

SPEC-DIFF #1 (forgetting is coupled to re-observation). The specification states that decay happens "every window" and that "a cell that stops being observed decays out." In code, the entire log-odds-and-clamp block of §IV-B is inside if (touched). An untouched cell — out of the sensor field of view, with neither a hit nor a miss that window — does not decay; its ℓ freezes. Forgetting therefore requires being re-observed as free (a miss is a touch), so clutter fades only while it stays in the field of view, and anything that leaves the field of view persists indefinitely. This is the most consequential difference and is restated as an operational limitation in the discussion.

SPEC-DIFF #2 (periodicity guards absent from the formal block). The specification's §3.4 pseudocode omits the periodicity guards entirely, listing graduate as !graduated && $p \geq \text{graduate_prob}$ && observations $\geq \text{min_observations}$ and demote as graduated && $p \leq \text{demote_prob}$. The !periodic and || periodic terms actually in the code (§IV-C) appear only in prose and comments. A method transcribed verbatim from that block would be wrong; the equations above follow the code.

SPEC-DIFF #3 (amplitude gate counts touched windows, not elapsed time). The specification says amplitude is zero before period_windows windows have elapsed. The code gates on $n < T$, where n is the count of touched windows (calls to gather), not elapsed wall or window time. A sparsely observed cell needs T touches, which can span far more than T windows — the mechanism behind the low-duty periodicity miss reported in the evaluation.

V. Evaluation (synthetic)

A. Setup

All four experiments (E1–E4) are driven by a standalone harness in paper/experiments/ that links directly against the strata_core sources — no ROS 2, no ament, no colcon. Reproduction from the repository root is a single call, bash paper/experiments/run_all.sh, which configures and builds strata_core and the four harness executables, runs each one, and writes the CSVs in paper/experiments/results/ from which every number and figure in this section is taken; strata_core's own ctest suite

(1/1 target, aggregating the 56 gtest cases) is checked first, so the harness is only trusted once the unit-level behavior it builds on is confirmed. Every stochastic experiment (E1–E3) derives its `std::mt19937` generator deterministically from the fixed constant `kSeed = 12345` (with per-backend/per-configuration offsets recorded in code), and the seed is logged in each CSV’s `#seed` row, so the reported precision/recall/F1/flicker/TPR/FPR numbers are exactly reproducible. All reported magnitudes are single-seed (12345) point estimates: no cross-seed variance is characterized, so figures such as 0.9253 vs. 0.9758 should be read as illustrative of a mechanism rather than as robust distributional estimates. E4 additionally reads the wall clock to time `integrate()/endWindow()` calls; its absolute microsecond figures are therefore machine-dependent, while the live-cell counts it also reports are deterministic (they follow only from ray geometry, not timing).

What is compared. There is no external baseline. The lifelong-SLAM systems of §II do not expose a persistence core that this harness could drive. The FreMEn per-cell model [2], [6], [14], with its amplitude-ranked component selection, could be driven by the same streams, but we have not run it, so every comparison below is internal.

The four experiments map onto the two thesis claims of §III–IV (C2, C5) differently. E1 and E4 probe backend-behavior equivalence: each backend is driven, in its native geometry, by its own deterministically seeded hit stream — Grid2DBackend on the 2D integer cell lattice, Voxel3DBackend in continuous 3D, with per-run seeds `kSeed+offset` recorded in code — and we compare the outcomes. Because the classifier both backends call is the same LayeredMap type, any divergence arises in the geometry layer (point→CellId mapping and the free-space ray walk) together with its native input sampling; this is a native-geometry comparison, not a controlled same-input isolation. E2 and E3, by contrast, drive LayeredMap/PeriodicityModel directly, with no backend in the loop at all — a choice that is itself an argument for C1: because persistence, hysteresis, and periodicity are implemented once, independent of CellId provenance, characterizing them against LayeredMap directly is valid for whichever backend supplies the cell ids in production. Each harness constructs its own LayeredMapParams in code rather than loading the shipped YAML, so several values deviate from the Table IV production defaults; Table V lists every deviation, per experiment, so the runs are reproducible without reading the harness source. Three deviations are common to interpreting the results below: all four harnesses run at `layer_interval` 1 (one window per integration tick, against the production default of 10), so “window” and “frame” coincide in every reported count; E1, E3, and E4 disable periodicity to isolate the persistence layer (E2 is the only run that exercises the FreMEn model); and E4 and E2 both set `survival_decay` 1.0 (no forgetting). E2 in particular raises all six of the log-

TABLE V
Per-experiment deviations from the Table IV production defaults. “—” = default unchanged; “swept” = varied across the E3 configuration grid ($\lambda \in \{0.90, 0.97, 1.0\}$, `graduate_prob` $\in \{0.6, 0.7, 0.8, 0.9\}$, `demote_prob` $\in \{0.3, 0.5, \text{graduate_prob}\}$); “n/a” = inert because periodicity is disabled in that run.

Parameter (default)	E1	E2	E3	E4
<code>layer_interval</code> (10)	1	1	1	1
<code>enable_periodicity</code> (true)	false	true	false	false
l_{hit} (0.85)	—	1.0	—	—
l_{miss} (−0.4)	—	−1.0	—	—
<code>survival_decay</code> (0.97)	—	1.0	swept	1.0
<code>graduate_prob</code> (0.8)	—	0.9	swept	—
<code>demote_prob</code> (0.45)	—	0.4	swept	—
<code>min_observations</code> (3)	—	5	—	—
<code>prune_prob</code> (0.05)	—	—	0.01	—
<code>period_windows</code> (24)	n/a	8	n/a	n/a
<code>n_harmonics</code> (2)	n/a	3	n/a	n/a

TABLE VI
E1 — final static-set precision/recall/F1, both backends, all three clutter densities.

backend	density	recall = 1	prec. @39	F1 @39
grid2d	low (5/w)	window 3	1.0000	1.0000
grid2d	med (25/w)	window 3	0.9938	0.9969
grid2d	high (100/w)	window 3	0.9253	0.9612
voxel3d	low (5/w)	window 3	1.0000	1.0000
voxel3d	med (25/w)	window 3	0.9938	0.9969
voxel3d	high (100/w)	window 3	0.9758	0.9877

odds/graduation defaults it touches (l_{hit} 1.0, l_{miss} −1.0, λ 1.0, p_{grad} 0.9, p_{dem} 0.4, N_{min} 5) to drive its detected-versus-pruned classification, and E3 lowers `prune_prob` to 0.01 to keep noisy walls alive; these are the values actually compiled into each harness.

B. E1 — Static-layer map quality vs. time

Setup. A ground-truth static cross of 161 cells (segment $y=60, x \in [20, 100]$ and $x=60, y \in [20, 100]$) is hit every window, alongside transient movers at three densities — low/med/high = 5/25/100 fresh random cells per window — over 40 windows, run against both a 120×120 (resolution 1.0) Grid2DBackend and a 1.0-voxel Voxel3DBackend, with periodicity disabled to isolate the persistence layer. Static-set precision/recall/F1 against the 161-cell wall set is logged per window (results/e1_static_quality.csv).

With $N_{\text{min}} = 3$, a cell needs three touched windows before it is eligible to graduate; the CSV confirms recall jumps from 0 to 1.0 exactly at the third window (window index $t=2$) for every one of the six backend×density combinations, and holds at 1.0 through $w=39$ in every case — no wall cell is ever demoted by clutter. The only errors are a handful of false-positive statics that appear when a random mover happens to re-hit the same cell often enough to graduate on its own; at low density (5 movers/window) this never happens at all,

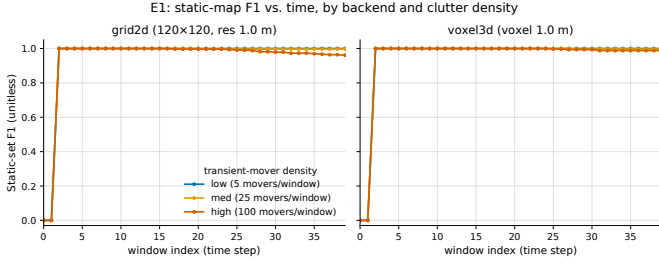


Fig. 2. E1: static-set F1 vs. window index, one panel per backend, one line per clutter density. Both backends recover $F1 \approx 1.0$ within three windows; the 100-movers/window regime is the only one with a visible precision/F1 dip, and the dip is larger for grid2d.

and precision stays exactly 1.0 through $w=39$ for both backends. At medium density it saturates at a single spurious cell (precision floors at 0.9938 and never falls further), but at high density (100 movers/window) false statics keep accumulating window over window — from 162 predicted cells at $w=17$ (grid2d) / $w=25$ (voxel3d) to 174 (grid2d) / 165 (voxel3d) by $w=39$ — giving the only material gap between backends: precision 0.9253 (grid2d) vs. 0.9758 (voxel3d) at the final window. Both backends run the identical LayeredMap graduation rule, each on its own independently seeded native mover stream — Grid2DBackend drawing movers on the integer cell lattice, Voxel3DBackend drawing them continuously, both confined to a single z -plane here — so the small precision gap reflects native quantization and sampling differences rather than the classifier. We do not claim it isolates a single geometric mechanism.

C. E2 — Periodicity detection

Setup. Three periodic doors (p8 4-on/4-off, p8 2-on/6-off, p4 2-on/2-off, all with periods commensurate with the FreMEN base period $T=8$ using $H=3$ harmonics), a constant wall, and four deterministic Bernoulli(0.5) aperiodic movers as negative controls, are driven through the real LayeredMap pipeline for 64 windows. Reported: per-cell final class against ground truth (Periodic TPR/FPR), and FreMEN dominant-harmonic amplitude vs. observation length (results/e2_classification.csv, results/e2_amplitude_vs_length.csv, results/e2_summary.csv).

Both 50%-duty doors are detected cleanly: amplitude 0.653 and 0.707 clear $a_{\min} = 0.3$ by a wide margin. The one miss, the 25%-duty door (door_p8_2on6off), is a prune/maturity race, not a modeling failure: the notes accompanying this harness report that the real pipeline prunes this cell in roughly 10 of its 64 windows — each 6-window vacant stretch lets its log-odds fall below p_{prune} while its FreMEN amplitude is still 0 (the $n \geq T$ gate has not yet been reached), erasing its accumulator state before periodicity can mature. The CSV's ref_amplitude column — a non-pruning reference PeriodicityModel fed the identical occupancy stream — reads 0.46194 for this same cell, comfortably above $a_{\min}$,

TABLE VII
E2 — per-cell classification against ground truth ($a_{\min} = 0.3$); Periodic TPR = 2/3, FPR = 1/5.

cell	ground truth	final class	ampl.	note
door_p8_4on4off (50% duty)	periodic	Periodic	0.653	correct
door_p4_2on2off (50% duty)	periodic	Periodic	0.707	correct
door_p8_2on6off (25% duty)	periodic	Transient	—	miss
wall_constant	non-periodic	Static	≈ 0	correct
aperiodic_1	non-periodic	Periodic	0.329	FP
aperiodic_0,_,_3	non-periodic	Transient	0.14–0.25	correct

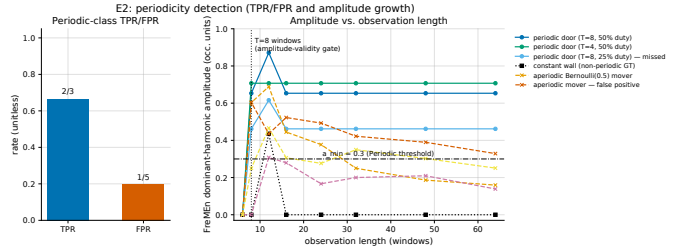


Fig. 3. E2: (left) Periodic-class TPR/FPR bars; (right) FreMEN dominant-harmonic amplitude vs. observation length for all 8 probe cells, with the $a_{\min} = 0.3$ threshold and the $n \geq T$ validity gate marked — the two 50%-duty doors clear the threshold cleanly, the 25%-duty door is pruned before it matures, and one aperiodic mover produces the false positive.

confirming the signal itself is periodic and detectable; the miss is specifically an interaction between pruning and the touched-window amplitude validity gate. The lone false positive, aperiodic_1 at amplitude 0.329, is a Bernoulli(0.5) cell whose Fourier energy over 64 windows happens to clear the 0.30 threshold by a small margin (0.029 above $a_{\min}$) — the kind of chance alignment expected from a fixed-length random negative control rather than a systematic classifier flaw. The amplitude-vs-length data also confirms the validity gate operates as specified: for door_p8_4on4off, amplitude is exactly 0 at observation length 6 ($n < T=8$) and jumps to 0.653 at length 8 ($n \geq T$), where it then stabilizes.

D. E3 — Sensitivity: hysteresis band and decay

Setup. 50 ground-truth wall cells are observed occupied with probability 0.6 each window (noisy) alongside 10 fresh movers/window, over 80 windows, with the identical noise sequence replayed for every configuration in the sweep. The sweep spans $\text{graduate_prob} \in \{0.6, 0.7, 0.8, 0.9\} \times \text{demote_prob}$ (a low floor 0.3, a mid floor 0.5, and the degenerate $\text{demote_prob} == \text{graduate_prob}$, i.e. zero hysteresis band) $\times \text{survival_decay} \in \{0.90, 0.97, 1.0\}$ — 36 configurations total. Reported: final Static-layer F1

TABLE VIII

E3 — selected sweep configurations (full 36-row sweep in the CSV); identical replayed noise throughout.

grad.	dem.	band	decay	flicker	F1
0.9	0.9	0	0.90	574	0.810
	(deg.)				
0.8	0.8	0	0.90	343	0.947
	(deg.)				
0.7	0.7	0	0.90	250	0.980
	(deg.)				
0.9	0.9	0	0.97	150	1.000
	(deg.)				
0.9	0.9	0	1.00	122	1.000
	(deg.)				
0.9	0.3	0.6	0.90	54	1.000
0.9	0.3	0.6	0.97	52	1.000
0.9	0.3	0.6	1.00	52	1.000

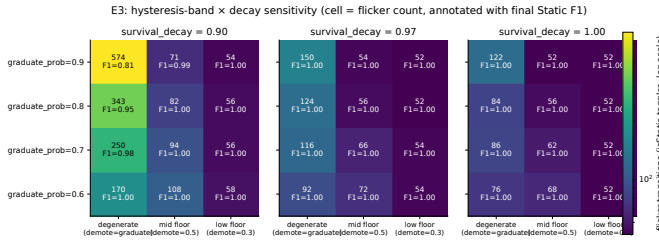


Fig. 4. E3: three heatmap panels (one per survival_decay); rows = graduate_prob, columns = hysteresis-band width, cells colored by flicker-transition count (log scale) and annotated with the raw count and final F1 — the degenerate (zero-band) column drives both the worst flicker and the worst F1 at every decay setting.

and the count of per-window isStatic flicker transitions (results/e3_sensitivity.csv).

Removing hysteresis is the single largest effect in the sweep: at graduate_prob = demote_prob = 0.9, survival_decay = 0.90 (the worst degenerate configuration), the wall layer flickers 574 times and loses 19% of its final F1 (0.810, recall collapses to 0.68), while a same-noise, same-decay run with the hysteresis band widened to 0.6 (demote_prob = 0.3) flickers only 54 times at F1 1.000 — the band alone accounts for a $> 10\times$ reduction in flicker. The degenerate (zero-band) configurations are also internally monotonic in threshold: at survival_decay = 0.90, flicker rises with the coincident threshold value (250 at 0.7, 343 at 0.8, 574 at 0.9) even though a higher threshold is ordinarily the more conservative, more “static” setting — a threshold pair with no gap between graduate and demote is not a substitute for a hysteresis band, regardless of where that pair sits. survival_decay is the second-order effect: holding the worst degenerate threshold pair (0.9/0.9) fixed, flicker falls monotonically as forgetting slows — 574 at $\lambda=0.90$, 150 at $\lambda=0.97$, 122 at $\lambda=1.0$ — because faster decay lets noisy log-odds cross the (here, coincident) threshold more often. With a wide hysteresis band, survival_decay barely matters (52–

TABLE IX

E4 — per-call latency, throughput, live-cell count, and estimated memory, both backends, all three extents. Absolute μ s are single-machine wall-clock samples; live-cell counts (and the ratios derived from them) are geometry-determined and reproducible.

backend	extent	μ s/intg.	updates/s	live cells	memory
grid2d	100 ²	320.8	9.41×10^6	8,553	479 KB
voxel3d	100 ²	4,796.8	7.06×10^6	85,399	4.78 MB
grid2d	250 ²	1,702.6	1.07×10^7	45,657	2.56 MB
voxel3d	250 ²	18,469.5	4.46×10^6	295,300	16.5 MB
grid2d	500 ²	6,548.9	8.64×10^6	150,096	8.41 MB
voxel3d	500 ²	35,958.3	4.45×10^6	661,410	37.0 MB

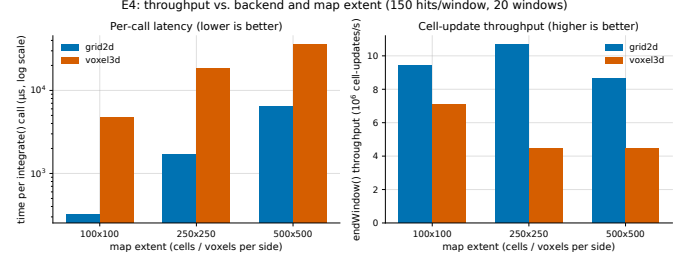


Fig. 5. E4: grouped bars by backend $\times$ extent — per-integrate() latency in μ s (log scale, left) and endWindow() cell-update throughput in 10^6 updates/s (right). grid2d is cheaper per call than voxel3d at every tested extent; the gap tracks voxel3d’s larger live-cell count.

54 flicker across all three decay settings at the best band): hysteresis, not decay, is what keeps the static layer stable.

E. E4 — Throughput and memory

Setup. 150 random hits per window over 20 windows, for Grid2DBackend vs. Voxel3DBackend at three nominal extents (100/250/500, resolution/voxel size 1.0), periodicity disabled. Reported: mean per-integrate() call latency, mean per-endWindow() cell-update rate, final live-cell count, and estimated memory at the fixed 56 B/cell footprint documented in §IV (CellEvidence 24 B + ≈ 32 B unordered_map node overhead, with FreMen storage excluded since periodicity is off in this run) (results/e4_throughput.csv). The committed E4 timings predate grid2d’s per-cell flag byte (§IV-F), which adds a one-byte write per cleared ray cell and per hit to integrate(); the cost ratio has not been re-measured since.

At every tested extent, Grid2DBackend’s integrate() call is cheaper than Voxel3DBackend’s at the identical nominal extent and hit rate — by $4,796.8/320.8 \approx 15.0\times$ at 100², $18,469.5/1,702.6 \approx 10.8\times$ at 250², and $35,958.3/6,548.9 \approx 5.5\times$ at 500², the multiple shrinking as extent grows. The live-cell counts explain why: Voxel3DBackend ends each run holding $85,399/8,553 \approx 10.0\times$, $295,300/45,657 \approx 6.5\times$, and $661,410/150,096 \approx 4.4\times$ as many cells as Grid2DBackend at the same extent, because its free-space ray-sample march (§IV-F) walks a full 3D volume and spawns far more traversed voxels than Grid2DBackend’s exact 2D Bresenham clear over the

same nominal footprint — the same geometric asymmetry that drives the latency gap also drives the cell-count gap, and both narrow together as the extent grows and the ray paths (and hence the per-ray voxel counts) become a smaller fraction of the total volume. Memory is not measured: the harness multiplies the live-cell count by the fixed 56 B/cell estimate (evidence and hash node with periodicity off; table and backend overhead excluded), so estimated memory tracks live-cell count across every row of Table IX by construction, topping out at 37.0 MB for the largest 3D configuration tested (661,410 cells $\times$ 56 B) — inexpensive even at the largest map in this sweep. Absolute microsecond values here are single-run wall-clock measurements on one machine and are not claimed to be portable; the live-cell counts, and the ratios and memory figures derived from them, follow only from ray/hash geometry and reproduce under the fixed seed regardless of machine.

F. Synthesis

Across E1–E4, the two backends behave as one engine wearing two geometries, exactly as C1 asserts by construction: E1’s near-identical F1 trajectories (both backends graduate the wall by the third window and hold recall 1.0 through $w=39$) and E4’s live-cell counts are the empirical face of “no per-dimension logic or cost,” since LayeredMap and PeriodicityModel — exercised directly and identically in E2 and E3 — never see which backend produced the CellId they are updating. The two places where the backends do diverge trace to native geometry rather than to the shared classifier — which E2 and E3 exercise directly, backend-free, so it is provably identical for both backends: E1’s high-clutter precision gap (0.9253 vs. 0.9758) reflects the backends’ different native mover sampling (grid2d on the integer cell lattice, voxel3d continuously, both at a single z -plane in E1) rather than a demonstrated z -separation mechanism, and E4’s per-call latency gap (5.5–15.0 $\times$, narrowing with extent) follows from Voxel3DBackend’s free-space ray-sample march spawning 4.4–10.0 $\times$ more live voxels than Grid2DBackend’s exact line clear at the same nominal extent. E2 and E3, run once against LayeredMap with no backend involved at all, characterize the shared engine’s own behavior — clean 50%-duty periodicity detection with a diagnosed prune/maturity-race failure mode at low duty cycle, and hysteresis as the dominant flicker suppressor, decay a secondary one — and that characterization is, by the same construction argument, valid for whichever backend feeds it cell ids in production.

VI. Discussion and Limitations

A. What has and has not been shown

The evaluation in §V characterizes a shipped engine; it does not validate a deployed system. Every number in this paper — E1’s precision/recall curves, E2’s TPR/FPR, E3’s flicker counts, E4’s throughput and

memory — comes from a deterministic, seeded synthetic harness compiled directly against `strata_core`, driving hand-authored hit/miss patterns through the real LayeredMap/PeriodicityModel code. No real sensor, no real robot motion, and no field deployment enters any of these results. This is a statement of present scope, not a hidden gap: ELite [8], LT-mapper [9], the long field deployment of Berrio et al. [13] and the FreMen grid of Krajník et al. [6] all report long-term real-world evidence that STRATA does not yet have. No external baseline is run, not even a FreMen model driven by the same streams, so this paper does not show that STRATA’s periodicity test labels periodic cells better than FreMen’s amplitude-ranked component selection, or that the classifier is useful on real sensor data. The evidence establishes a transparent state machine on synthetic streams, not a reliable four-class scene classifier: in E2 the low-duty door is lost to pruning and one aperiodic control is labelled Periodic. STRATA is, so far, validated the way a library is validated — by unit and characterization tests against known ground truth — not the way a robot behavior is validated, by miles driven.

A second, narrower scope limit sits inside the “life-long” framing itself. STRATA implements single-session temporal persistence: a cell’s class is a running function of its own log-odds, observation count, and Fourier accumulators, updated window by window within one continuous mapping run. There is no session boundary, no re-localization into a prior map, and no cross-session registration — a graduated Static cell simply is the current map, with its history implicit in accumulator state rather than explicitly stored. This is a strictly smaller claim than LT-mapper’s live/meta/delta session management or Yang et al.’s versioned base-map-plus-deltas [20], both of which let a robot query or reconstruct a past session. STRATA cannot do either; it only ever has “now.”

Finally, the paper does not claim bit-identical cross-backend behavior. §V already frames grid2d and voxel3d as equivalent to within geometry- and native-input-induced differences rather than identical, and E1’s clutter-driven precision gap (0.9253 vs. 0.9758 at 100 movers/window) and E4’s 5.5–15.0 $\times$ per-integrate() cost gap are exactly those differences, measured rather than glossed over.

B. The out-of-FOV forgetting caveat is an operational limitation, not a footnote

§IV-H documents SPEC-DIFF #1 as an implementation fact: decay is applied only inside the touched branch of `endWindow()`, so a cell that leaves the sensor’s field of view stops decaying entirely — its log-odds value freezes at whatever it last reached, rather than relaxing toward the unknown prior. Restated as an operational consequence: a transient object that is observed long enough to accumulate strong occupied evidence, and then permanently exits the sensor’s coverage (a box dragged out of a corridor the robot no longer revisits, a door

propped open just outside the LiDAR’s usual sweep), retains its evidence indefinitely. STRATA’s forgetting is coupled to re-observation, not to elapsed time. In a bounded, frequently re-swept environment — the setting E1–E4 implicitly assume, and the setting the canonical Integration.WallStaticMoverTransientDoorPeriodic scenario exercises — this rarely matters, because clutter that matters is clutter the robot keeps driving past. In a sparsely-revisited environment it is a real failure mode the current implementation does not address, and it is one of the concrete items in the future-work list below rather than an incidental detail.

Free space is remembered, not forgotten. The same coupling shows in grid2d’s free value. A cell that a ray has cleared reads free (0) while its last observation was free and no Periodic, Static or occupied-leaning Transient class claims it, so free means “last observed free, with no current occupied evidence”, not “currently free”. An obstacle that appears where the sensor no longer looks reads free until it is observed; once observed it reads 50 from its first hit and keeps reading 50 out of view until a ray observes the cell free again. The bit costs one byte per grid cell, bounded by the grid size. voxel3d publishes no free space.

Scan-counted windows depend on the input rate. With window_mode scans a window is layer_interval integrated messages, so its length in seconds, and with it the period that period_windows stands for, follows the rate of messages the node actually integrates. Messages lost on a best-effort link, dropped for missing TF or skipped by an overloaded node stretch every window. In the synthetic ROS check, withholding 40% of the scans turns every tested door cell static with scan windows (grid2d), while time windows keep the door periodic in grid2d and out of the static map in voxel3d; CI asserts all three. Time windows trade this for evidence that depends on the rate: a window with few messages is less likely to see a hit, so a sensor rate that varies with the phase of a cycle can itself look periodic, outside the null of §IV-D. Once a second, on a wall-clock timer, the node publishes on /diagnostics the input rate, the share of messages lost, the TF failures, the window duration and the periodic period in seconds. The loss is estimated per sensor frame from gaps between stamps against the 20th-percentile interval, which holds while at least about a fifth of consecutive messages arrive back to back; above about 80% loss, or whenever the sensor rate is known, set expected_scan_rate_hz. The node warns when scan windows lose more than rate_warn_drop_fraction or jitter, when time windows go empty or get under half the expected messages, or when stamps stop advancing, and turns ERROR after input_timeout_s without input. A deployment should measure these on the robot (scripts/rate_report.py) and set window_period_s to layer_interval over the sensor rate. The experiments of §V integrate every message and are unaffected.

C. Design trade-offs owned honestly

STRATA makes three simplifications relative to richer prior formalisms, each adopted deliberately and each with a known cost.

A global scalar decay rate, not a per-cell-learned one. survival_decay is a single tunable constant shared by every cell, applied as in §IV-B. The Persistence Filter’s survival-time posterior [15] and the Markov-chain occupancy models of Tipaldi et al. and Saarinen et al. [25], [26] instead let each cell (or region) learn its own forgetting rate from observed transition statistics — a wall in a rarely-disturbed alcove and a doormat in a busy doorway would decay at different, empirically fit rates. STRATA’s one-parameter simplification is what E3 characterizes: a single survival_decay value, uniformly applied, already buys most of the achievable stability once paired with hysteresis (F1 1.000 for the wide-band configuration at both decay=0.90 and decay=1.0), so the paper does not claim the uniform rate is a limitation in the tested regime — only that it is a strictly less expressive model than the per-cell-learned alternative, and a scene with sharply heterogeneous dynamics per cell is untested.

A flat voxel hash, not an octree. Voxel3DBackend stores live voxels in an unordered_map<CellId, CellEvidence> keyed by a packed int64, with no multi-resolution compression. OctoMap [31] and UFOMap [33] instead exploit spatial coherence — large free or occupied regions collapse into single octree nodes — trading a more complex tree structure for asymptotically better memory scaling on sparse or large-extent maps. E4 measures the cost of this choice directly: voxel3d holds 4.4–10.0× the live-cell count of grid2d at matched scene extent because every sampled free-space point along a ray becomes its own hash entry, and the estimated per-cell footprint is a flat 56 B with periodicity off, regardless of spatial redundancy. The trade is deliberate — O(1) insert/lookup with no tree-rebalancing logic keeps the backend simple enough to unit-test the way §III describes — but it means STRATA’s voxel3d backend will not scale as gracefully as an octree-backed map to very large, sparsely-occupied 3D extents.

Two backends behind an if/else, not a plugin registry. §III argues, and the Related Work rejection of pluginlib [37] restates, that a compile-time MapBackend interface with two concrete unique_ptr members selected by one runtime string parameter is the right amount of mechanism for exactly two backends. This is a design bet, not a measured result: it costs nothing today, but it does not generalize — a third backend (e.g. a signed-distance or hybrid representation) would need either a third branch bolted onto the same if/else (tolerable once, ugly repeated) or a migration to pluginlib-style dynamic loading. The paper takes no position on which is correct beyond two backends; it only argues the current structure is not premature generalization at the current backend count.

Honesty has a cost the paper pays openly. Choosing the systems/tool framing over an accuracy-SOTA framing means every claim in §V is scoped to "what the shipped code measurably does," not "how well STRATA localizes or maps relative to a competing system." No baseline comparison against OctoMap, ELite, LT-mapper, or FreMen is run in this paper. The lifelong-SLAM systems do not expose a persistence core that this harness could drive; the FreMen per-cell model could be driven by the same streams, but we have not run it (§V). The E1–E4 harness is offered instead as a replayable protocol (§VII) a future study could run against an alternative implementation, rather than a leaderboard result against one run today.

D. When not to use STRATA

STRATA’s boundaries, stated plainly: it is not a substitute for SLAM — it performs no pose estimation, scan matching, or loop closure, and it consumes rather than produces the map $\rightarrow$ sensor transform, so a deployment without an external localizer (prism_loc or otherwise already publishing that TF) has no path to a usable map. It is not a substitute for multi-session mapping — a robot that needs to compare Monday’s map against Friday’s, or replay a specific past session, needs LT-mapper- or Yang et al.-style session versioning [9], [20] that STRATA does not provide. And it is not a substitute for semantic or object-level reasoning — the cell-class state machine of §IV-E has no notion of "this cluster of cells is one dynamic agent," unlike Khronos’s scene graph [11] or ERASOR’s per-object removal [30]. And it does not discover periods: where the relevant cycles are unknown or are not harmonics of one base period, FreMen’s selection among candidate periods [2] or warped hypertime [29] is the better tool. Within those boundaries — a single continuous mapping session behind an external localizer, at the raw occupancy-cell level, in 2D or 3D — STRATA is the intended fit.

E. Future work

Four items follow directly from the limitations above, in the order they would be tackled: (1) a FreMen baseline on the same synthetic streams and a four-class confusion under the shipped defaults, then validation on at least one real 2D and one real 3D LiDAR sequence, and real-robot and multi-session field validation, closing the synthetic-only gap of §VI.A, on both a 2D wheeled platform and a 3D-LiDAR platform to exercise both backends under real sensor noise; (2) a per-cell-learned decay rate along the lines of iMac/Tipaldi [25], [26], replacing the global survival_decay scalar characterized above with region- or cell-adaptive forgetting; (3) resolving the prune/maturity race identified in E2 — the 25%-duty periodic door is pruned during its own vacant stretches before its FreMen amplitude has enough touched-window support to mature (§V) — by either exempting recently-active cells from pruning or feeding a running amplitude estimate into the prune decision itself; and (4) decoupling forgetting from

re-observation, i.e. fixing SPEC-DIFF #1 so an untouched cell’s log-odds relaxes toward the unknown prior on a wall-clock or tick basis rather than only ever decaying when re-touched, closing the out-of-FOV persistence gap without changing the touched-cell update path E1–E4 already characterize.

VII. Conclusion

STRATA packages known ingredients of lifelong occupancy mapping — log-odds accumulation, Persistence-Filter-style survival decay, Removert-motivated Schmitt-trigger hysteresis, ReFusion-style negative evidence from free-space ray clearing, and FreMen-lite periodicity detection at one configured period — into one geometry-free LayeredMap/PeriodicityModel engine that drives two pluggable geometries, a 2D occupancy grid and a 3D voxel hash, behind a single MapBackend interface and a one-parameter runtime switch. The only code that differs per backend is point-to-cell-id mapping and the free-space ray walk (§III–§IV); the classifier that decides Static, Periodic, Transient, or Unknown is written once and shared by composition. That engine builds and unit-tests with a plain C++17 + Eigen toolchain, independent of ROS 2, DDS, or PCL (§III), and its behavior is characterized — not merely asserted — by a seeded, reproducible synthetic harness: near- identical static-layer quality across backends up to a clutter-induced, geometrically-explained precision gap (E1), clean detection of 50%-duty periodicity with a diagnosed failure mode at low duty cycles (E2), hysteresis as the dominant stabilizer against flicker (E3), and a flat estimated per-cell memory footprint with periodicity off, with a 5.5–15.0 $\times$ backend cost gap attributable to 3D free-space voxel proliferation, not to the shared engine (E4). Its closest prior work, the FreMen spatio-temporal grid of Krajník et al. [6], has long-term real-world evidence that STRATA lacks; a FreMen baseline on the same streams and real LiDAR sequences are the next evaluation steps. None of this replaces field validation, a comparison with an external baseline, multi-session mapping, period discovery, or semantic reasoning — §VI states those boundaries explicitly — but within them (synthetic evidence, no external baseline, a single session, a fixed base period, an external localizer), STRATA is offered as a small, dependency-light module other systems can sit on top of rather than a competing full lifelong-SLAM pipeline.

A. Reproducibility

STRATA is open source under the Apache-2.0 license at github.com/kjungmo/strata, targeting ROS 2 Humble. The strata_core package alone builds and unit-tests without ROS installed (`cmake -S strata_core -B build -DSTRATA_CORE_BUILD_TESTS=ON && ctest, §III`); the continuous-integration workflow builds and tests this core with a C++17 toolchain (g++, Eigen3) on ubuntu-latest. All figures and tables in §V regenerate from the CSVs the harness itself writes:

bash paper/experiments/run_all.sh configures and builds the four E1–E4 executables against strata_core, runs them with the fixed harness seed kSeed = 12345, and writes paper/experiments/results/*.csv; a follow-up paper/experiments/plot/run_all_plots.sh regenerates every figure from those CSVs. To reproduce or audit the parameterization, the engine parameters and production defaults are in Table IV (§IV-G) and the per-experiment deviations from those defaults are in Table V (§V). strata_core’s own ctest suite (1/1) is checked before the harness output is trusted. We invite integration reports, alternative backend implementations replayed against the Integration.WallStaticMoverTransientDoorPeriodic reference scenario, and pairing with an external localizer such as prism_loc for end-to-end deployment.

References

- [1] P. Biber and T. Duckett, “Dynamic maps for long-term operation of mobile service robots,” in *Robotics: Science and Systems I (RSS)*, Cambridge, MA, USA, 2005.
- [2] T. Krajník, J. P. Fentanes, J. M. Santos, and T. Duckett, “FreMen: Frequency map enhancement for long-term mobile robot autonomy in changing environments,” *IEEE Transactions on Robotics*, vol. 33, no. 4, pp. 964–977, 2017.
- [3] T. Krajník, J. P. Fentanes, G. Cielniak, C. Dondrup, and T. Duckett, “Spectral analysis for long-term robotic mapping,” in 2014 IEEE International Conference on Robotics and Automation (ICRA), Hong Kong, China, 2014, pp. 3706–3711.
- [4] D. Arbuckle, A. Howard, and M. J. Matarić, “Temporal occupancy grids: a method for classifying the spatio-temporal properties of the environment,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, vol. 1, Lausanne, Switzerland, 2002, pp. 409–414.
- [5] N. C. Mitsou and C. S. Tzafestas, “Temporal occupancy grid for mobile robot dynamic environment mapping,” in 2007 Mediterranean Conference on Control & Automation, 2007, pp. 1–8.
- [6] T. Krajník, J. Pulido Fentanes, M. Hanheide, and T. Duckett, “Persistent localization and life-long mapping in changing environments using the frequency map enhancement,” in 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2016, pp. 4558–4563.
- [7] T. Krajník, J. Santos, B. Seemann, and T. Duckett, “FROctomap: An efficient spatio-temporal environment representation,” in *Advances in Autonomous Robotics Systems (TAROS 2014)*, ser. Lecture Notes in Computer Science. Springer, 2014.
- [8] H. Gil, D. Lee, G. Kim, and A. Kim, “ELite: Ephemerality meets LiDAR-based lifelong mapping,” in 2025 IEEE International Conference on Robotics and Automation (ICRA), 2025.
- [9] G. Kim and A. Kim, “LT-mapper: A modular framework for LiDAR-based lifelong mapping,” in 2022 IEEE International Conference on Robotics and Automation (ICRA), 2022.
- [10] M. Labbé and F. Michaud, “RTAB-map as an open-source lidar and visual SLAM library for large-scale and long-term online operation,” *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.
- [11] L. Schmid, M. Abate, Y. Chang, and L. Carlone, “Khronos: A unified approach for spatio-temporal metric-semantic SLAM in dynamic environments,” in *Robotics: Science and Systems (RSS)*, 2024.
- [12] T. Kim, G. Kang, T. I. Kim, S. Song, and H. K. Ko, “MTD-Map: Single-stage long-term LiDAR map maintenance framework via mixture transition distribution,” *arXiv:2606.29469*, 2026, accepted to IROS 2026.
- [13] J. S. Berrio, S. Worrall, M. Shan, and E. Nebot, “Long-term map maintenance pipeline for autonomous vehicles,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 10 427–10 440, 2021.
- [14] STRANDS Project, “FreMen: Frequency map enhancement, source code repository,” <https://github.com/strands-project/fremen>, 2017, last commit December 2017; accessed 2026-09-30.
- [15] D. M. Rosen, J. Mason, and J. J. Leonard, “Towards lifelong feature-based mapping in semi-static environments,” in 2016 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2016, pp. 1063–1070.
- [16] G. Kim and A. Kim, “Remove, then revert: Static point cloud map construction using multiresolution range images,” in 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020, pp. 10 758–10 765.
- [17] E. Palazzolo, J. Behley, P. Lottes, P. Giguère, and C. Stachniss, “ReFusion: 3d reconstruction in dynamic environments for RGB-D cameras exploiting residuals,” in 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2019, pp. 7855–7862.
- [18] D. Meyer-Delius, J. Hess, G. Grisetti, and W. Burgard, “Temporary maps for robust localization in semi-static environments,” in 2010 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Taipei, Taiwan, 2010, pp. 5750–5755.
- [19] F. Pomerleau, P. Krüsi, F. Colas, P. Furgale, and R. Siegwart, “Long-term 3D map maintenance in dynamic environments,” in 2014 IEEE International Conference on Robotics and Automation (ICRA), 2014, pp. 3712–3719.
- [20] L. Yang, S. M. Prakhya, S. Zhu, and Z. Liu, “Lifelong 3d mapping framework for hand-held & robot-mounted LiDAR mapping systems,” *IEEE Robotics and Automation Letters*, vol. 9, no. 11, pp. 9446–9453, 2024.
- [21] E. Stefanini, E. Ciancolini, A. Settini, and L. Pallottino, “Efficient 2D LIDAR-based map updating for long-term operations in dynamic environments,” in 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2022, pp. 832–839.
- [22] —, “Safe and robust map updating for long-term operations in dynamic environments,” *Sensors*, vol. 23, no. 13, p. 6066, 2023.
- [23] X. Chen and B. Sun, “Layer assignment for long-term LiDAR map maintenance using geometric and semantic evidence,” *Remote Sensing*, vol. 18, no. 14, p. 2385, 2026.
- [24] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [25] G. D. Tipaldi, D. Meyer-Delius, and W. Burgard, “Lifelong localization in changing environments,” *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1662–1678, 2013.
- [26] J. Saarinen, H. Andreasson, and A. J. Lilienthal, “Independent Markov Chain occupancy grid maps for representation of dynamic environments,” in 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2012, pp. 3489–3495.
- [27] M. Saavedra-Ruiz, S. B. Nashed, C. Gauthier, and L. Paull, “Perpetua: Multi-hypothesis persistence modeling for semi-static environments,” *arXiv:2507.18808*, 2025, accepted to IROS 2025.
- [28] T. Krajník, J. M. Santos, and T. Duckett, “Life-long spatio-temporal exploration of dynamic environments,” in 2015 European Conference on Mobile Robots (ECMR), 2015, pp. 1–8.
- [29] T. Krajník, T. Vintr, S. Molina, J. P. Fentanes, G. Cielniak, O. M. Mozos, G. Broughton, and T. Duckett, “Warped hyper-time representations for long-term autonomy of mobile robots,” *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3310–3317, 2019.
- [30] H. Lim, S. Hwang, and H. Myung, “ERASOR: Egocentric ratio of pseudo occupancy-based dynamic object removal for static 3d point cloud map building,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 2272–2279, 2021.
- [31] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, “OctoMap: An efficient probabilistic 3d mapping framework based on octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [32] M. Nießner, M. Zollhöfer, S. Izadi, and M. Stamminger, “Real-time 3d reconstruction at scale using voxel hashing,” *ACM Transactions on Graphics*, vol. 32, no. 6, pp. 169:1–169:11, 2013.

- [33] D. Duberg and P. Jensfelt, “UFOMap: An efficient probabilistic 3d mapping framework that embraces the unknown,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 6411–6418, 2020.
- [34] S. Macenski, D. Tsai, and M. Feinberg, “Spatio-temporal voxel layer: A view on robot perception for the dynamic world,” *International Journal of Advanced Robotic Systems*, vol. 17, no. 2, 2020.
- [35] D. V. Lu, D. Hershberger, and W. D. Smart, “Layered costmaps for context-sensitive navigation,” in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Chicago, IL, USA, 2014, pp. 709–715.
- [36] S. Macenski and I. Jambrecic, “SLAM toolbox: SLAM for the dynamic world,” *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.
- [37] Open Robotics, “Creating and using plugins (c++) — ROS 2 documentation,” <https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Pluginlib.html>, 2024, accessed 2026-07-02.
- [38] J. E. Bresenham, “Algorithm for computer control of a digital plotter,” *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965.